Heartwise / Cardio Signal: A Reproducible, Interpretable Decision-Tree Pipeline for Heart-Disease Risk Screening and Web Deployment

Sections 1?3

Author: Ayush Maurya

Date: 2 November 2026

1. Introduction

1.1 Motivation

Cardiovascular diseases (CVDs) account for an estimated 17.9 million deaths each year, making them the leading cause of mortality globally [1]. Early identification of people at elevated risk can support lifestyle counselling, further investigation, and timely treatment. Statistical and machine-learning risk models have therefore been studied for decades, from the Framingham risk equations to modern gradient-boosted and deep-learning models. Yet the gap between a model that performs well in a notebook and a model that can be responsibly used, inspected, and maintained in a real setting remains large.

Three problems recur in applied student and research projects:

1. **Reproducibility.** Data cleaning, imputation, encoding, scaling, and model selection are often scattered across notebook cells, with inconsistent category orders and undocumented assumptions.
2. **Deployment.** A serialized Python pipeline is difficult to serve from a conventional web stack, and pickle files are sensitive to library-version drift.
3. **Communication.** A probability produced by a tree leaf is not necessarily a calibrated personal risk, and a binary label such as "heart disease" can be misread as a diagnosis.

This paper studies a project that attempts to address all three concerns in a small, self-contained system.

1.2 The Project

The project is stored in the repository `heart_prediction`; the web application lives in the folder `TitanicPredictionWeb` and is an ASP.NET Core 8 Razor Pages application. A user enters ten routine clinical measurements: age, sex, chest-pain type, resting blood pressure, cholesterol, fasting blood sugar, resting electrocardiogram (ECG) result, maximum heart rate, exercise-induced angina, and ST-slope category. The application encodes and scales these values, traverses an embedded decision tree, displays a risk score and a flag, and writes the input and result to a MySQL table.

The repository also contains the original exploratory notebook (`predict_heart_disease.ipynb`), a small inference notebook (`predict_heartpart2.ipynb`), the legacy serialized scikit-learn pipeline (`pipe.pkl`), the combined dataset (`heart.csv`), and a retraining/export script (`train_heart_model.py`). The deployed model is not loaded from `pipe.pkl`; it is a shallow, smoothed tree retrained by `train_heart_model.py` and exported to `TitanicPredictionWeb/Models/heart-disease-model.json`, a 1,785-byte file embedded into the .NET assembly.

The project uses several names: the folder is `TitanicPredictionWeb`, the assembly and namespace are `HeartHealthWeb`, the web brand is `Cardio Signal`, the README calls the product `Heartwise`, and the database is `titanic_db`. This naming debt is documented in Section 6.10 as a maintainability issue rather than hidden.

1.3 Objectives

The project and this paper pursue the following objectives:

1. Build a complete, reproducible pipeline from a public CSV dataset to a running web application.
2. Use a model simple enough to be inspected and explained to a non-specialist.
3. Avoid the binary 0%/100% outputs of a fully grown tree by limiting depth and smoothing leaf probabilities.
4. Keep the production runtime independent of Python and scikit-learn.
5. Persist screening inputs and outputs in a relational database without destroying pre-existing data.
6. Evaluate the deployed model honestly, including its limitations and its comparison with the legacy pipeline.
7. Identify the engineering, ethical, and regulatory work that would be required before clinical use.

1.4 Contributions

The main contributions of the work are:

- **An end-to-end reference implementation** that connects data preparation, model selection, JSON export, ASP.NET Core inference, and MySQL persistence in one repository.
- **A leakage-aware preprocessing protocol** in which imputation, encoding, and scaling are fitted inside each cross-validation fold.
- **A systematic model-selection procedure** covering 70 tree configurations and six smoothing strengths, selected by log loss rather than accuracy alone.
- **A compact, language-agnostic model contract** that represents a fitted scikit-learn tree as arrays and scalars, allowing exact inference in C# without a Python runtime.
- **A critical empirical comparison** between the deployed shallow tree and the legacy fully grown pipeline, including cross-validation, hold-out behaviour, calibration, and ablations for omitted or unused features.
- **A candid account of limitations**, including dataset bias, missing-value handling, non-calibrated leaf scores, diagnostic wording in the user interface, and the absence of automated tests and external validation.

1.5 Scope and Non-Goals

This is an educational and engineering case study. It is **not** a clinical trial, a medical device, a diagnostic system, or a substitute for professional medical advice. The model is trained on a public, retrospective, de-identified dataset. No new patient data were collected. The system has not been externally validated, prospectively evaluated, or reviewed by a regulatory body. The paper therefore treats predictive performance as an internal, descriptive result rather than a claim of clinical utility.

2. Background

2.1 Clinical and Epidemiological Context

Coronary artery disease and other cardiovascular conditions are multifactorial. Well-established risk factors include older age, male sex, elevated blood pressure, dyslipidaemia, diabetes or impaired fasting glucose, smoking, obesity, physical inactivity, and a family history of premature CVD [1, 2]. Clinical presentations vary from asymptomatic disease to stable angina, unstable angina, and myocardial infarction. Chest-pain type, resting ECG findings, exercise-induced angina, maximum heart rate, and ST-segment slope are commonly used in non-invasive assessment and exercise testing.

The dataset used here encodes several of these factors. It does not include smoking, diabetes (only fasting blood sugar), body mass index, family history, or medication use. A model trained on such a limited feature set can only capture a partial picture. This is one reason the application is framed as an educational demonstration rather than

a screening programme.

2.2 The UCI Heart Disease Data

The original UCI Heart Disease dataset was assembled by Detrano and colleagues and donated in 1988 [3, 4]. It combines data from four institutions: the Cleveland Clinic Foundation, the Hungarian Institute of Cardiology in Budapest, the University Hospital in Zürich, and the V.A. Medical Center in Long Beach, California. The UCI version contains 920 records and 76 attributes, although most studies use a 14-attribute subset. The target is usually a binary indicator of angiographic coronary artery disease, derived from a stenosis severity threshold.

The `heart.csv` file in this project has 918 rows and 12 columns: eleven predictors and one binary target. This is the widely circulated "Heart Failure Prediction Dataset" derivative distributed on Kaggle [5]. It is derived from the same combined UCI sources but is not identical to the raw UCI distribution: it may exclude a small number of records, it recodes the field names, and it presents the target as `HeartDisease` with values 0 and 1. The project's README and model metadata describe it as the "combined UCI heart disease dataset (918 records)". That description is accurate at the level of provenance, but users should consult the original UCI documentation and the Kaggle dataset page for exact licensing and preprocessing details before redistributing the data.

2.3 Decision Trees for Clinical Prediction

A decision tree recursively partitions a feature space into rectangular regions. At each internal node, a feature and threshold are chosen to maximise a purity criterion; at each leaf, a prediction is stored. For classification, common criteria are Gini impurity and entropy. The resulting model is attractive in clinical and educational settings because it can be displayed as a set of if-then rules, audited by a clinician, and explained without post-hoc approximation [6, 7, 20].

Trees also have well-known weaknesses. A fully grown tree can memorise training data, produce extremely confident leaves, and change substantially under small perturbations. Single trees are high-variance estimators. These problems are usually addressed by pruning, depth limits, minimum leaf sizes, ensembling (random forests, gradient boosting), or smoothing. This project deliberately chooses a shallow single tree to preserve interpretability, and compensates for the loss of flexibility with depth and leaf-size constraints plus Beta-prior smoothing.

2.4 Probability, Calibration, and Decision Thresholds

A classifier's output can be a class label or a score. A score is not automatically a probability. Calibration refers to agreement between predicted probabilities and observed event rates: among patients assigned a score of 0.8, approximately 80% should experience the event if the score is well calibrated [8, 9]. Calibration is important because clinical decisions depend on absolute risk, not only on ranking.

Common metrics reflect different aspects of performance:

- **Discrimination** (ROC AUC) measures the probability that a randomly chosen positive case receives a higher score than a randomly chosen negative case.
- **Log loss** penalises confident wrong predictions and is sensitive to probability quality.
- **Brier score** is the mean squared error of probabilistic predictions.
- **Accuracy, sensitivity, specificity, positive predictive value (PPV), and negative predictive value (NPV)** describe behaviour at a chosen threshold.

A model can rank well but be poorly calibrated, or be accurate at one threshold but clinically unhelpful at another. The application uses a fixed 0.5 threshold for the displayed flag, but the underlying score is continuous. The project explicitly warns that the displayed score is a leaf class share, not a personal clinical probability. This distinction is central to the paper's discussion.

2.5 Deployment and Reproducibility

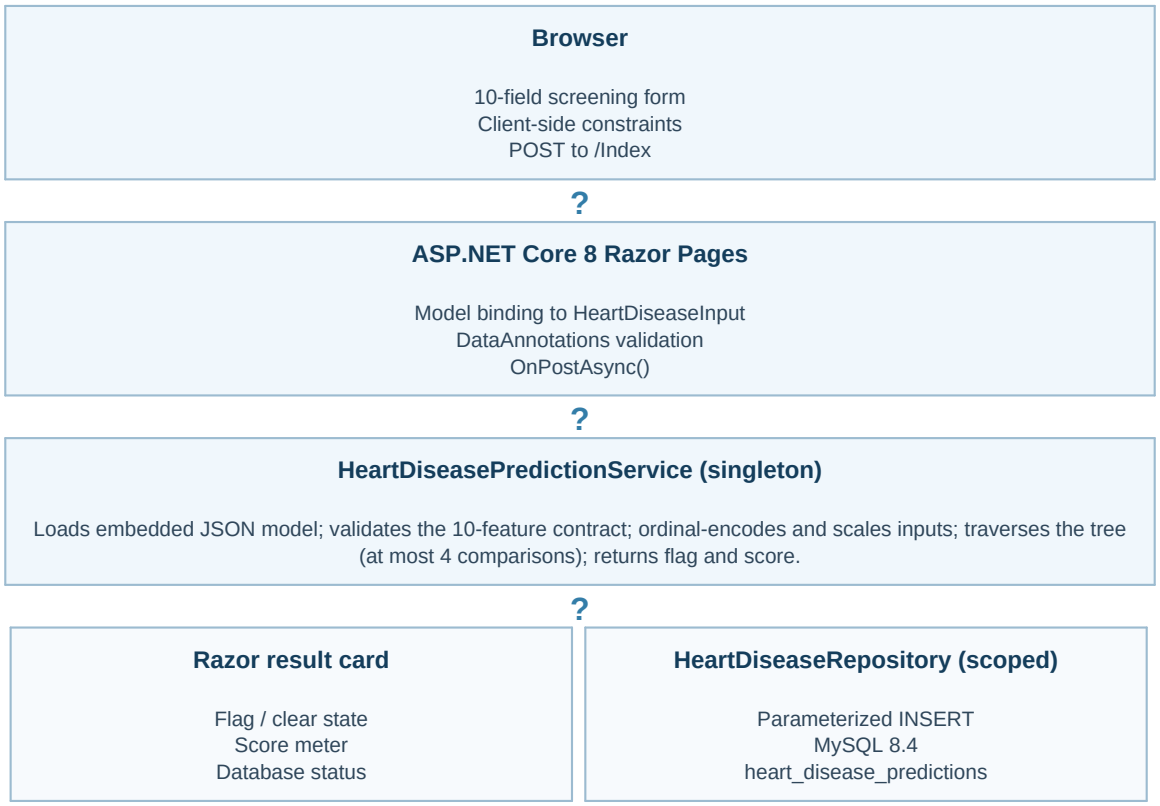
Deploying a model is an engineering activity, not only a modelling one. A production system needs a stable inference contract, deterministic preprocessing, dependency management, input validation, error handling, persistence, and a way to audit what was predicted. Serialized Python objects are convenient for experimentation but fragile across library versions. The project's answer is to export the fitted tree's structure and preprocessing parameters to JSON, embed that JSON in a .NET assembly, and implement the traversal in C#. This removes the Python runtime from the serving path and makes the model contract explicit.

The broader reproducibility literature emphasises complete reporting of data sources, preprocessing, model selection, performance, and limitations [10, 11]. This paper follows that spirit: it reports not only the selected model and its metrics but also the alternatives that were considered, the features that were omitted, the legacy pipeline that was replaced, and the engineering controls that are still missing.

3. System Overview

3.1 Architecture

The application is a conventional server-rendered web app with a single prediction endpoint implemented as a Razor Page. The flow is:



The model is computed before persistence. If MySQL is unavailable, the user still sees the model result and a clear message that the record could not be saved. This separation is a deliberate resilience choice.

3.2 Technology Stack

Layer	Technology	Version / notes
Runtime	.NET	8.0
Web framework	ASP.NET Core Razor Pages [17]	`Microsoft.NET.Sdk.Web`
Language	C#	Nullable and implicit usings enabled

Layer	Technology	Version / notes
Data access	MySqlConnector	2.6.2
Database	MySQL	8.4.13 portable local installation in the development environment
Model training	Python	3.13 virtual environment
Modelling library	scikit-learn [16]	1.9.1 in the current `.venv`; legacy `pipe.pkl` reports 1.6.1
Data handling	pandas / NumPy	3.1.0 / 2.5.4
Model artifact	JSON	1,785 bytes, embedded resource
IDE integration	VS Code tasks	Start website; initialise database
Host OS	Windows	Paths and PowerShell script are Windows-specific

The production application does **not** need Python, pandas, NumPy, or scikit-learn. The Python environment is used only to retrain and export the model.

3.3 Request Lifecycle

1. A `GET /` request renders `Pages/Index.cshtml` with an empty `HeartDiseaseInput` populated by default values.
2. The user submits the form with `POST /`.
3. Razor Pages binds form values to `IndexModel.Patient` and runs DataAnnotations validation.
4. If validation fails, the page is re-rendered with field-level and summary errors; no prediction is made.
5. If validation succeeds, `HeartDiseasePredictionService.Predict` encodes, scales, and traverses the tree.
6. The page model attempts to persist the input and result through `HeartDiseaseRepository.SaveAsync`.
7. A successful insert sets `SavedToDatabase = true`; a MySQL or configuration error is logged and surfaced as a user-readable message.
8. The page is re-rendered with the result card, score meter, flag, and database status.

3.4 Repository Layout

Path	Purpose
`heart.csv`	918-row combined UCI heart disease dataset
`predict_heart_disease.ipynb`	Original exploratory notebook and legacy pipeline construction
`predict_heartpart2.ipynb`	Small notebook that loads `pipe.pkl` and predicts one profile
`pipe.pkl`	Legacy serialized scikit-learn pipeline (245 nodes, 123 leaves)
`train_heart_model.py`	Retraining, model selection, JSON export, and round-trip verification
`TitanicPredictionWeb/TitanicPredictionWeb.csproj`	Web project; assembly name `HeartHealthWeb`
`TitanicPredictionWeb/Program.cs`	Application bootstrap and dependency injection
`TitanicPredictionWeb/Models/HeartDiseaseInput.cs`	Input DTO with validation attributes

Path	Purpose
`TitanicPredictionWeb/Models/HeartDiseasePredictionResult.cs`	Output record `(HasHeartDisease, ModelScore)`
`TitanicPredictionWeb/Models/heart-disease-model.json`	Embedded tree and preprocessing parameters
`TitanicPredictionWeb/Services/HeartDiseasePredictionService.cs`	Runtime inference engine
`TitanicPredictionWeb/Services/HeartDiseaseRepository.cs`	Parameterised MySQL insert
`TitanicPredictionWeb/Pages/Index.cshtml`	Screening form and result UI
`TitanicPredictionWeb/Pages/Index.cshtml.cs`	Page handler and orchestration
`TitanicPredictionWeb/Pages/Error.cshtml`	User-friendly error page
`TitanicPredictionWeb/wwwroot/css/site.css`	Dark, responsive visual design
`TitanicPredictionWeb/database/init.sql`	Database and table creation
`TitanicPredictionWeb/Start-HeartHealth.ps1`	Starts MySQL if needed, prompts for password, runs the app
`.vscode/tasks.json`	VS Code tasks for starting the app and initialising the database